

Chap 4

0 系列只能在末尾写, 即使 fseek, 写还是会跳到末尾

mode

mode

meaning

"r"

只读, 文件必须已开

"w"

只写, 若文件已会清空, 不存在会创建开

"a"

追加写, 文件指针在文件末尾, 不会创建末

"r+"

读写, 文件必须已, 允许读和写, 文件指针在开头开

"w+"

读写, 若文件已会清空, 不会创建开

"a+"

读写追加, 文件指针在末尾, 允许读写读-开写-末

对于二进制文件, 需在模式中加 'b', 告诉库不要做换行符或字符编码转换

"rb"

只读二进制

"wb"

只写二进制

其余同上

"ab"

追加二进制

"r+b" "rbt"

读写二进制

"w+b" "wbt"

读写二进制

"a+b" "abt"

读写追加二进制

fseek 一般用于二进制
fread 制 file, 若文件 file
fwrite 结构型, 也可用

Sequential & Random Access

1. 顺序访问: 必须从前往后读文件 O(n) 只用于文本

常见顺序读取函数: fscanf, fgets, getc, fread (顺序用时)

2. 随机访问: 可直接跳到文件某个位置读取

(只用于 核心函数: fseek)

binary file) 固定长度记录 + fseek 定位 = 可直接 insert / modify / delete 单条记录

functions fread() & fwrite()

size_t is an unsigned integer (通常用于表示: 内存大小, 对象 num, 数组大小)

1. size_t fread() 存放读取数据的内存地址 要读取的数据数量

size_t fread(void* Data, size_t ObjSize, size_t NumObjs, FILE* indata);

每个对象大小 (字节)

文件指针

It returns the # of objects (not bytes) actually read, which may be fewer than NumObjs if the end of file is met.

eg. int arr[5];

fread(arr, sizeof(int), 5, fp); 读取 5 个 int, 每个 int 大小 sizeof(int)

从 fp 读取存入 arr.

文件 → 内存变量

这两个类型不可乱变, 如果你发现变了一定是 fread, fwrite 写错了

2. size_t fwrite() 从这个地址开始拿数据

size_t fwrite(const void* Data, size_t ObjSize, size_t NumObjs, FILE* outdata);

it returns the # of objects (not bytes) actually written, which may be fewer than NumObjs if the end of file is met.

A'ZONE

eg. `int arr[5];`

`fwrite(arr, sizeof(int), 5, fp);` 写入5个int, 每个int大小 `sizeof(int)`,
从arr读取数据写入文件
内存变量 → 文件
空字符串, 也就是长度为0的字符序列, 但仍
占有一个终止符 '\0' 的空间.

3. 实例

`struct clientData blankClient = {0, "", 0.0};`
表示空账户

✓

`for (i=1; i<=100; i++)`

这儿写的是数据类型...

`fwrite(&blankClient, sizeof(struct clientData), 1, outdata);`

在文件中创建100个空账户, 可直接用随机访问方式定位和写入. → 创建存储空间.
(在调用 `fread` 或 `fwrite` 时, 文件的位置指针会自动移动(更新))

✗

`fwrite(&blankClient, sizeof(struct clientData), 100, outdata);`

若只有一个结构体变量, 严格来说, *the function reads beyond the valid memory region of the variable, the behavior is undefined and may result in writing garbage data to the file.*

四. File positioning

file.

`fseek` → sets the file position pointer to a specific locations in the
then `fwrite` → writes the data.

1. `fseek()`

已打开的文件指针

偏移量(字节)

偏移基准

`int fseek(FILE* stream, long int Offset, int Origin)`

eg. 假设文件内容是37 int

(3 → 页 ←)

`[int1] [int2] [int3] [EOF]`
`0x3 4x7 8x11`

Origin

Measure offsets from

Beginning of file

Current position

End of file

`fseek(fp, 0, SEEK_END)` `SEEK-SET`

指向的是位置12 `SEEK-CUR`

因此 `-n * sizeof(int)` 正好
把指针放在倒数第n个数 `SEEK-END`

的起始地址 `fseek(fp, (5-1) * sizeof(struct clientData), SEEK-SET);`

第5条开始

`r+b`: 可用 `fread` 读, 也可用 `fwrite` 写, 不清空文件. 打开文件时, 不会 delete 原内容

文件指针默认指向文件开头; 写入会 cover 当前位置数据. 不会自动追加到文件末尾

2. more on `fread` `fwrite`

每次调用 `fread`, `fwrite`, 文件内部的位置指针会自动向后移动. 下次读/写会从上

一次结束的位置开始.

在顺序访问多条记录时, 无需 `fseek`


```
fseek(fp, offset, SEEK-SET);
```

```
fread (& 大小, n个, fp);
```

课堂讲义:

```
#include <stdio.h>
```

```
struct clientData {
```

```
    int acctNum;
```

```
    char lastname[15];
```

```
    char firstname[10];
```

```
    float balance;
```

```
};
```

```
int main() {
```

```
    struct clientData client;
```

```
    FILE * indata;
```

```
    int option, accountNum;
```

```
    if((indata=fopen("d:\\credit.dat", "r+b")) == NULL) {
```

```
        printf("File could not be opened.\n");
```

```
        return 1;
```

```
    }
```

```
    scanf("%d %c", &option);
```

```
    while (option != 1 || option != 2) {
```

```
        scanf("%d %c", &accountNum);
```

```
        fseek(indata, (accountNum-1)*sizeof(struct clientData), SEEK
```

```
        if(option == 1) {
```

```
            fread(&client, sizeof(struct clientData), 1, indata);
```

```
            if(client.acctNum == 0) printf("No information");
```

```
            else -- 吃回车, 若后面有%c, fgets, 不管理空格, '\n'
```

```
            } 防止"空字符串" 就会影响
```

```
        else {
```

```
            scanf("%d",
```

```
            clientNum = accountNum;
```

```
            fwrite(&client, sizeof(struct clientData), 1, indata);
```

```
        }
```

```
        scanf("%d %c", &option);
```

```
    }
```

```
    return 0;
```

```
}
```


2. Additional useful functions

1. `rewind()` `rewind(cindata);`

功能: 把文件位置指针移动到文件开头, 而不必关闭再重新 `fopen`

等价于 `fseek(stream, 0, SEEK_SET)`, 但还会清除文件错误 & EOF

2. `fflush()` 用了 `fwrite` 后就可使用这个

`fwrite` 后马上 `fflush(FILE * stream)`

`fread` 中间需对于 `fprintf(outdata, "%d %d %d", a1, a2, a3);`

用 `fflush()` `fflush(outdata);`

数据通常先进入内存缓冲区, 并不会写入磁盘文件。

只有在以下情况才会真正写入文件。

1) 缓冲区满

2) `fclose()`, `fseek()`, `frewind()`

3) 程序正常结束

4) 手动用 `fflush()`

几个需说明的点: 都已经让指针从头开始了, 无需在前加 `rewind`

1. `fseek(..., SEEK_SET)`

2. 那种文件读入小心人名中间有空格, `%s` 会读不完

eg. `char name[10];` $\Rightarrow$ Δ : 最多读 5 个字符

`%15[0-9]` 读取最多 15 个字符, 直到遇到数字为止, 且读进来的必须都不是数字

`%[^\n]` 读到换行前为止

`%c...` 按字符集合规则读

文件指针

会读 '\n'

并自动补

'\0'

`fgets(str, n, fp);`

存放读取内容的字符数组 最多读取的字符数包括 '\0'

$\rightarrow$ `fgets` 会最多读取 `n-1` 个字符, 并自动在最后加一个 '\0'

成功: 返回 `str`
失败: `NULL`

`fscanf` 返回: 成功匹配并赋值的变量个数 否则返回 `EOF(-1)`

3. } text file: `fscanf fgets fprintf` 也会被当真 $\Rightarrow$ 死循环

binary file: `fseek fread fwrite`

4. `while(fscanf(cindata, "%d %*c", &temp, rec-Num) == 1)` 公马

5. `gets()` 只能从 `stdin` 读取数据, 且会一直读取 input 直到遇到 '\n' 才停止

6. `fgets()` 停止条件

① 一旦读到 '\n', 立即停止并把 '\n' 一起存进字符串 eg. 文件: `hello\nworld`

删 '\n': ① `str[strlen(str, '\n')] = '\0';` ② `len = strlen(str); str[len-1] = '\0';` `str = "hello\n"`

② 读满 `n-1` 个字符, `n` 留信 '\0'

③ 遇 EOF

`fgets` 从当前文件指针位置开始读 前面的空格, '\n' 处
不会像 `%s` 一样空白字符跳过 理干净